



**National Centre for
Atmospheric Science**

NATURAL ENVIRONMENT RESEARCH COUNCIL



**University of
Reading**

Data catalogue support for cf-python

Background to, and early plans/progress for, data catalogue access

Sadie Bartholomew
CMS Weekly Meeting, 2026-03-27

I (of 3). Context: data(set) access

Data access in the CF Data Tools: current status

- At present `cf-python` (and `cfdm` to some extent) support reading of, in various formats, **local (on disk) datasets** and for some formats those available remotely either via an OPeNDAP (Open-source Project for a Network Data Access Protocol) server URL or via an Amazon S3 cloud object storage schema (S3-compatible service), e.g:

```
>>> f = cf.read("../my_datasets_dir/file.nc") # relative or absolute filesystem path
>>> g = cf.read("https://psl.noaa.gov/thredds/dodsC/Datasets/file.nc.dds") # OPeNDAP URL
>>> h = cf.read("s3://store/data/file.nc") # S3 path
```

- From the `cf.read` API reference (<https://ncas-cms.github.io/cf-python/function/cf.read.html>):

“The following file formats are supported: netCDF, CDL, Zarr, PP, and UM fields file. NetCDF and Zarr datasets may be on local disk, on an OPeNDAP server, or in an S3 object store. CDL, PP, and UM fields files must be on local disk.”

Other stores of relevant data for potential access routes

We want to bring (all) the ARD to the people for loading (as effortlessly and quickly as possible)!*

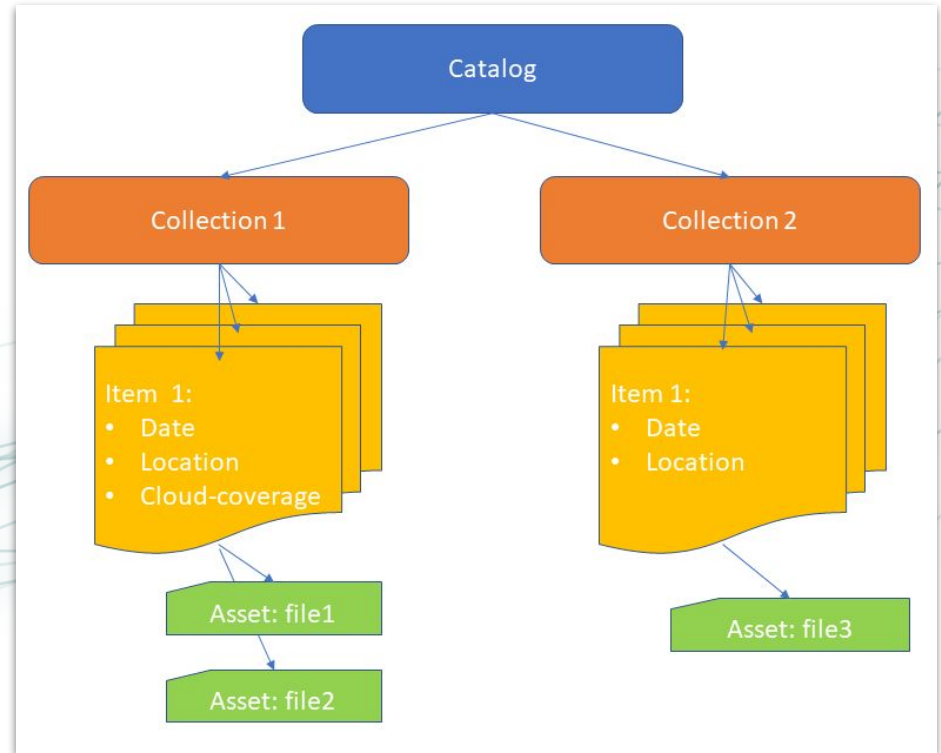
1. HTTP-based access beyond OPeNDAP e.g. THREDDS, CDNs and object storage over HTTPS
2. Other cloud object storage backends to S3 e.g. Google Cloud Storage (GCS), Azure Blob Storage
3. Data APIs and web services e.g. other RESTful APIs and OGC services such as WCS (Web Coverage Service)
4. Database-backed access: relational databases (PostgreSQL/PostGIS), array databases (e.g. SciDB, TileDB), or query engines (e.g. DuckDB, BigQuery)
5. Virtual/reference-based datasets (logical datasets assembled from multiple sources) e.g. kerchunk
6. **Data portals & catalog-driven access: e.g. intake and STAC catalogues, other federated data systems (e.g. ESGF-like architectures)**

The focus of this talk!

2 (of 3). Focusing in on catalog(ue)s

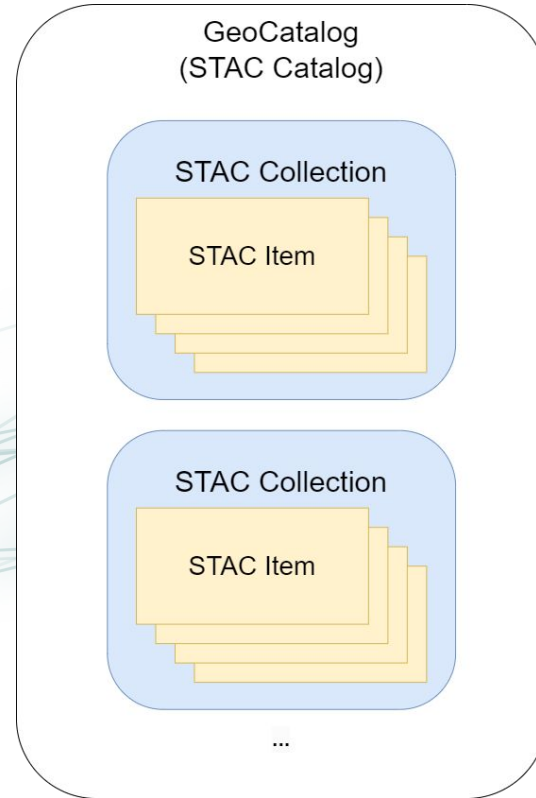
Both STAC and Intake center on concept of a catalog(ue)

- “A catalog is a collection of metadata records, each describing a resource, such as a dataset or service.” - the Open Geospatial Consortium (OGC definition)



I. STAC ($\equiv$ SpatioTemporal Asset) Catalogues

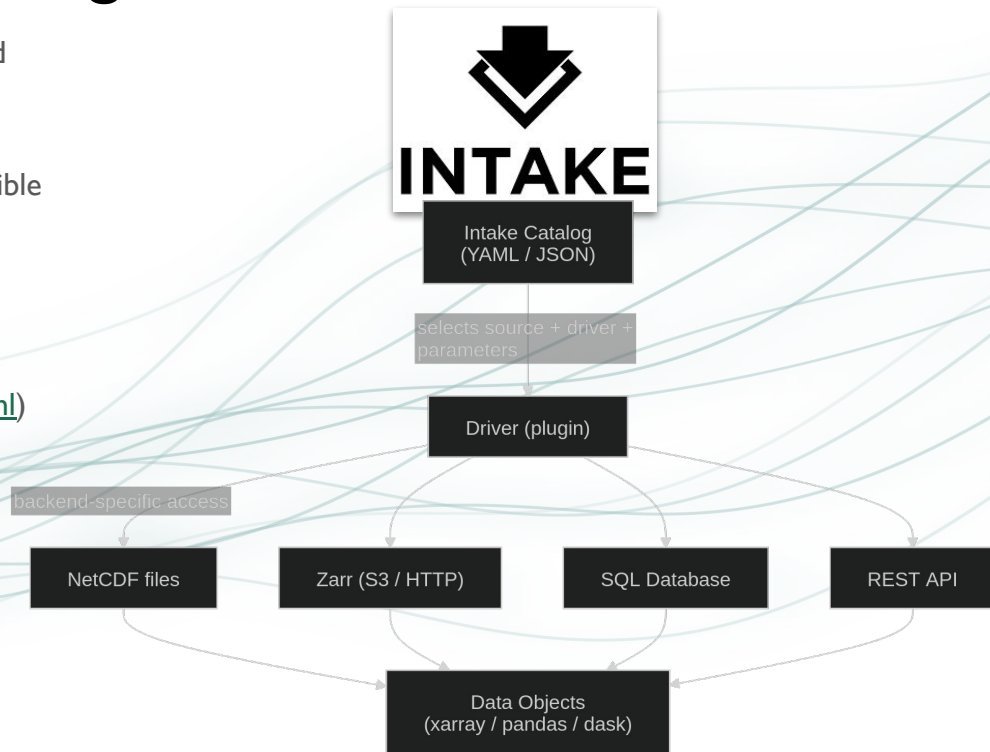
- They define a common metadata record specification and RESTful API for searching (by location, time, metadata filters) for searching through these
- They act as a universal index and access layer for earth/geospatial data
- Good resource for learning more is <https://stacspec.org/en/tutorials/intro-to-stac/>:
 - “STAC is a network of JSON files that reference other JSON files”
 - “three component specifications that together make up the core STAC specification: Item [Asset in diagram to left], Catalog, Collection”
- OSS tools for interfacing generally with STAC in Python include `pystac-client` (<https://pystac-client.readthedocs.io/en/stable/>) and `stackstac` (<https://stackstac.readthedocs.io/en/latest/>)



STAC API
Browse, Query, Download

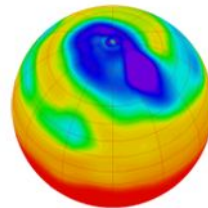
2. Intake: ecosystem for catalogue access abstraction

- Provides a consistent way to describe, discover, and load datasets from a wide range of storage backends and formats
- While STAC has a strict JSON schema, Intake has a flexible YAML or JSON based
- OSS tools for interfacing with Intake in Python include (where ESM, ESGF $\equiv$ the usual acronyms):
 - intake ('Take2' v2 in beta at present) (<https://intake.readthedocs.io/en/latest/index.html>)
 - intake-esm (<https://intake-esm.readthedocs.io/en/stable/>);
 - intake-esgf, "intake-esgf is an intake and intake-esm inspired package" (<https://intake-esgf.readthedocs.io/en/latest/>).



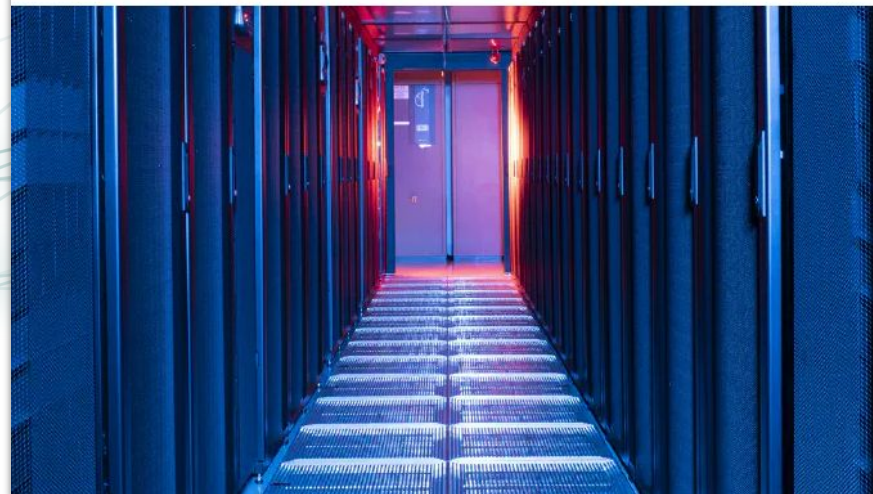
The CEDA Archive's STAC Catalogue

- CEDA has a STAC API which (at least as of mid 2025) is “still in a pre-production stage, subject to change as the collections are updated and metadata is revised”
- Can be accessed via the CEDA DataPoint library:
 - “DataPoint is a package developed at CEDA to provide key features for improved search functionality across the CEDA STAC catalog, as well as integrating known cloud-optimised formats for effective access to analysis-ready data”
- <https://stac.ceda.ac.uk> is a STAC Browser for exploring data on CEDA
- Good blog post by Dan Westwood on the topic at: <https://techblog.ceda.ac.uk/2025/07/24/ceda-datapoint.html>



Centre for Environmental Data Analysis

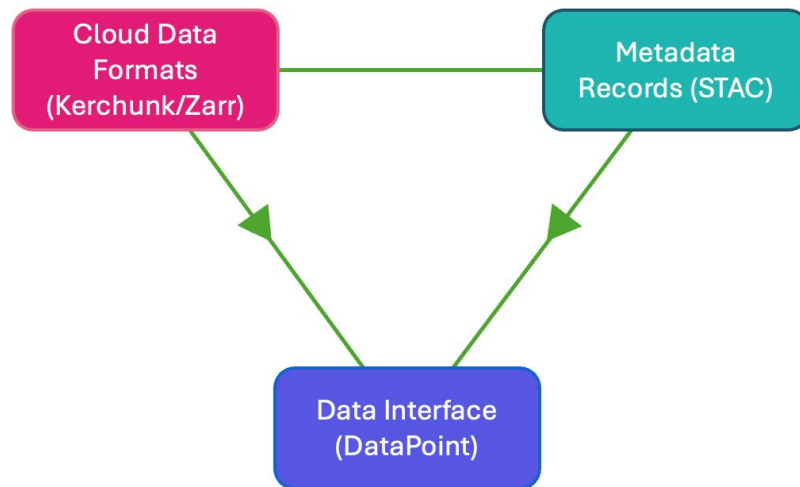
SCIENCE AND TECHNOLOGY FACILITIES COUNCIL
NATURAL ENVIRONMENT RESEARCH COUNCIL



CEDA DataPoint for Python-based STAC support

- CEDA-maintained Python API to access datasets in archive, with STAC-based metadata connections
- Can connect to *any* public STAC API - *not just the CEDA STAC API!*
- For more info, repo at: <https://github.com/cedadev/datapoint> and docs at: <https://cedadev.github.io/datapoint/>

The Future of Data Discovery at CEDA



3(of 3). Plans

Some discussion points RE cf-python catalogue support

- Ultimately we want to be able to load in data and do either a `cf.aggregate` or (ideally) `cf.concatenate` operation on it
- We want to support both STAC and Intake catalogues in cf-python.
 - Q) Which to tackle first?
- Regarding the STAC case, CEDA Datapoint looks useful!
 - Q) Do we utilise CEDA Datapoint or do we build standalone support into cf-python?
- The CEDA STAC catalogue holds a fair amount of CFA data!
 - Q) Can we use that in some way, since cf-python supports this natively?
 - Q) Can we use CF aggregation to make life easier?
- There are multiple Python tools for Intake (`intake`, `intake-esm`, and `intake-esgf`).
 - Q) Which of the existing tools do we build upon - or do we interface with some/all?

Plan going forward

- Use CEDA Datapoint library, because it has the same end goal and we can collaborate
- We will update it so that, on top of existing means ('mode') to open product datasets with xarray (as `xarray.Dataset` objects), they can be opened instead in cf-python (as `cf.Field` objects)
- So, there will be PR(s) to the <https://github.com/cedadev/datapoint> repo, see Issue: <https://github.com/cedadev/datapoint/issues/33>
- **We decided to tackle this (for now / as a way in) as 3 cases to deal with, separately:**
 1. Data is **in kerchunk format**: cf-python will shortly be able to read kerchunk, so this case is trivial (get, read, done)!
 2. Data is **in CFA format**: cf-python can already read CFA, so also trivial!
 3. **Neither of the above two are true**: the more general catch-all case. Get all files, sort them, and be able to load them as `cf.Field` objects.